

Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission

Abstract

Autonomous agents are promoted to production on the strength of what their execution traces say they did. Every deployed attestation scheme for such traces — in-toto’s runtime-trace predicate, SLSA, TUF, the 2026 agent-provenance drafts, and the per-action receipt schemes that have converged this year — shares a blind spot: the evidence collector is the sole observer, so an adversary who controls the recording path need not forge anything. It simply does not record. The resulting bundle is internally consistent, every signature verifies, and every integrity check passes, because nothing was tampered with.

This paper makes that gap precise and closes it. We define *Witnessed Trace Completeness*, a property relative to a declared set of witnesses — the tool endpoints, API gateways and sandbox brokers that actually serve agent actions. Each witness issues receipts carrying a monotonic per-session sequence number and, at session end, a signed commitment to how many actions it served. A release gate then *reconciles* rather than merely verifies. We prove that under EUF-CMA an adversary holding the agent and the collector, but no witness key, cannot omit a witnessed action; and we prove a *separation* result: no scheme whose verdict is a function of a single submitted document can achieve either witnessed completeness or replay-freedom, which places every record-format standard on the wrong side of a formal boundary rather than merely behind on features.

Against 6 omission attacks, in-toto/DSSE and TUF detect 0, per-issuer hash chaining 1, per-action receipts 2, and EVIASSURE without reconciliation 0; with reconciliation it detects 6/6, while an honestly witnessed execution is still approved. On the orthogonal tamper axis it blocks 16/17 vectors (94.1%, 95% CI [73.0, 99.0]) against 10 for an executed in-toto/DSSE + TUF + OPA composition, and we report the one vector it fails. A held-out evaluation over 1,075 profiles recalls 50/50 structurally overt behavioural anomalies and 0/25 semantically disguised

ones, marking where cryptographic release assurance stops and content-level policy must begin.

1 Introduction

Autonomous agents driven by Large Language Models (LLMs) act across tool environments, API endpoints and persistent memory stores [10, 13, 30]. Unlike a compiled artifact, whose behaviour is fixed by its code, an agent exhibits non-deterministic reasoning, dynamic tool invocation, and adaptive multi-step decision sequences. When such an agent is promoted to production — an updated checkpoint, a widened tool scope, a new pipeline stage — organisational security rests on verifying that safety invariants, authorization constraints and regression suites actually held *during that execution* [10].

Release pipelines verify no such thing. Deployment gates evaluate unauthenticated status signals — process exit codes, plain-text console logs, unsigned JSON build-status files — emitted by intermediate runner environments. An attacker who controls a build step, manipulates agent memory, or exploits a prompt-injection path can forge an approval signal outright, bypassing verification entirely [6, 23, 27, 29]. That such paths are exploited routinely is well established: taxonomies of documented supply-chain attacks enumerate hundreds of real incidents [16, 22], large-scale measurement finds package-manager compromise commonplace in interpreted-language ecosystems [8], and multi-repository update systems require dedicated defences [20].

The gap is one of subject rather than of strength. Supply-chain security attests the origin and integrity of *static* artifacts — container images, compiled binaries, model weight files — and does so well. But an agent’s security posture is not defined by its code artifact alone; it is defined by the dynamic, non-deterministic execution trace that artifact produced against a test suite. We can cryptographically attest *what* an agent is. We cannot, today, authorize a release on *how it behaved*.

To address these critical vulnerabilities, EviAssure¹ introduces an evidence-backed release assurance architecture that fills this niche. It replaces informal release signals with cryptographically authenticated Evidence Bundles backed by Merkle inclusion proofs, timestamp freshness windows, digital signatures, and provenance envelopes.

1.1 Key Contributions

The primary contributions of this work are:

1. **Witnessed Trace Completeness:** a security property, and a protocol achieving it, for the attack class every existing agent-attestation scheme leaves open — the collector that omits rather than forges. Sequence-bound witness receipts plus a signed closing count make omission require a witness-key forgery. We state the guarantee *relative to a declared witness set* and prove it (Theorem 1); actions outside that set remain undetectable, and we say so.
2. **A separation between record formats and authorization layers:** Theorem 2 shows that any scheme whose verdict is a function of one submitted document cannot achieve witnessed completeness or replay-freedom, for information-theoretic rather than engineering reasons. This converts a positioning argument that earlier drafts of this paper made rhetorically into a proof.
3. **Evidence-bound release authorization:** a fail-closed gate whose verdict is a deterministic, reproducible function of attested evidence under a pinned policy, with domain-separated (RFC 6962) depth-bound Merkle trace attestation and truncation-resistant count binding.
4. **A falsifiable evaluation, including where the system fails:** an omission suite with an honest-execution control, 17 tamper vectors of which five were authored against the design rather than the check list (three broke earlier revisions), executed in-toto/DSSE, TUF, hash-chaining and per-action-receipt baselines, clean negative controls, and a held-out inspection protocol. Every rate carries a Wilson interval, and we report one tamper vector and one detector blind spot that the system does not close.

Record Formats versus Authorization Layers. A distinction runs through this paper. in-toto provides

¹The complete anonymous source code, evaluation corpus, and benchmark suite are available at <https://anonymous.4open.science/r/eviassure-release-assurance/>.

signed, tamper-evident attestations of supply-chain steps, including vetted runtime-trace predicates [12], and SLSA builds graduated release provenance on top [27, 29]. These frameworks define *passive record formats*: they attest what occurred, but supply no *stateful authorization layer* that gates a release on a non-deterministic execution trace. EviAssure is that layer — an active, fail-closed gate — and Theorem 2 establishes that the distinction is formal rather than rhetorical. Concretely, EviAssure adds three enforcement mechanisms these specifications omit: (i) truncation-resistant trace binding via a signed trace count and, under Definition 5, witness reconciliation; (ii) freshness and replay protection held as gate *state* rather than asserted in a document; and (iii) a reproducible release decision under a pinned policy version. Composition is preserved: an **EvidenceBundle** is an in-toto v0.2 / SLSA v1.0 envelope, so existing records ride inside it unmodified. Section 8.3 compares against the 2026 agent-provenance standards in detail.

Open Science and Reproducibility. The full implementation, the 1,075-profile execution trace corpus, the 17-vector attack harness with its clean negative controls, and the executed baselines (in-toto/DSSE, TUF, and the Open Policy Agent (OPA) Rego policy) are released as a sanitized anonymous review artifact (Appendix C).

2 Preliminaries

2.1 Formal Security Definitions

Definition 1 (Unforgeability under Chosen Message Attack (EUF-CMA)). An Evidence Assurance Scheme $\Sigma = (\text{KeyGen}, \text{Package}, \text{Verify})$ is EUF-CMA secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a valid pair (E^*, σ^*) such that $\text{Verify}(PK, E^*, \sigma^*) = 1$ without E^* having been signed by an honest party holding SK is negligible in security parameter λ :

$$\Pr[\text{Exp}_{\Sigma, \mathcal{A}}^{\text{euf-cma}}(\lambda) = 1] \leq \text{negl}(\lambda) \quad (1)$$

Definition 2 (Merkle Trace Inclusion Soundness). For root R_M committed together with its leaf count N , and leaf H_i , a Merkle proof π_i is sound if no computationally bounded adversary can construct π'_i for any $H'_i \neq H_i$ — including internal node digests of the same tree — such that $\text{VerifyMerkle}(H'_i, \pi'_i, R_M, d) = 1$ for the committed depth $d = \lceil \log_2 N \rceil$, where verification accepts only proof paths of exactly length d and leaves and internal nodes are hashed under disjoint domain prefixes.

Definition 3 (Evidence Integrity). An evidence pack (R_M, σ, N) comprising a Merkle root R_M , signature σ , and signed trace count N provides *evidence integrity* if no

probabilistic polynomial-time adversary, given oracle access to the honest packager, can produce a verifying pack (R'_M, σ', N') that the packager never issued; equivalently, nothing that was *recorded* can be modified, reordered, inserted, or removed without detection. Integrity is a property of the signed record and is guaranteed by Theorem 3 below.

Definition 4 (Trace Completeness). Let E be an agent execution and let $\mathcal{C}$ be the evidence collector that observes E . A recorded trace set $T = \{T_1, \dots, T_N\}$ is *complete with respect to E* if every action of E that $\mathcal{C}$ observed is represented by exactly one T_i . Completeness is an observation property of the *collector*, not of the Merkle tree: the tree attests exactly the leaves it was given. If a real execution is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$ but the recorded trace is $A \rightarrow B \rightarrow D \rightarrow E$, the tree protects the recorded sequence perfectly and says nothing about C . No signature, hash chain, or inclusion proof over the submitted set can, because the adversary forged nothing — it declined to create evidence. Definition 5 makes this attainable by moving the observation out of the collector’s trust domain.

Definition 5 (Witnessed Trace Completeness). Let $\mathcal{W} = \{W_1, \dots, W_m\}$ be a set of *witnesses*, each holding a signing key the adversary does not control, and let $\text{med}(W_j)$ be the set of action types W_j mediates. An action is *witnessed* if its type lies in $\bigcup_j \text{med}(W_j)$. A submitted trace set T for session sid is *witness-complete* if for every $W_j \in \mathcal{W}$ there is a W_j -signed closing (sid, n_j) , the W_j -signed receipts in T are exactly the contiguous sequence $1, \dots, n_j$ for sid , each receipt binds to a distinct member of T , and every witnessed action in T carries a receipt.

Completeness is stated *relative to $\mathcal{W}$* : an action of a type no witness mediates is outside the guarantee. This is a deployment-controllable condition — mediate every externally observable effect — rather than the unbounded assumption that the collector observed everything.

Theorem 1 (Witnessed Completeness Soundness). Let $\mathcal{A}$ control the agent, the evidence collector and the collector’s signing key, but no key in $\mathcal{W}$. If the gate accepts a witness-complete bundle for session sid , then except with probability $m \cdot \text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}$, the trace set contains every witnessed action performed in sid .

Proof. Suppose $\mathcal{A}$ omits a witnessed action a served by W_j as its k -th action, and the gate accepts. W_j ’s counter is monotone and advances only on service, so W_j issued receipts $1, \dots, n_j$ with $k \leq n_j$, and its closing commits to n_j . Acceptance requires the receipt sequence for W_j to be exactly $1, \dots, n'_j$ with a matching closing. Three cases. (i) $n'_j = n_j$: then $\mathcal{A}$ supplied n_j distinct receipts, but only $n_j - 1$ correspond to submitted actions, so some receipt is

unmatched and reconciliation rejects — unless $\mathcal{A}$ minted a receipt, an Ed25519 forgery under W_j . (ii) $n'_j < n_j$: the presented closing differs from the one W_j signed, so $\mathcal{A}$ forged a closing under W_j . (iii) $\mathcal{A}$ substitutes a receipt from another session $sid' \neq sid$: the session identifier is inside the signed receipt payload, so the substitution is rejected without forgery. Each surviving case is a forgery against one of m witness keys; a union bound gives the stated advantage. Fabrication of a witnessed action is symmetric: the action carries no receipt, and minting one is again a forgery. $\square$

Theorem 2 (Record Formats Cannot Achieve These Properties). Let V be a *stateless document verifier*: a deterministic predicate $V(d) \in \{\text{accept}, \text{reject}\}$ over a single submitted document d , with access to fixed public parameters (trust roots, policy) but no state that varies with prior submissions and no channel to $\mathcal{W}$. Then V achieves neither replay-freedom nor witnessed completeness.

Proof. Replay-freedom. Determinism gives $V(d) = V(d)$: if V accepts d once it accepts every resubmission, so an adversary replays an approved release indefinitely. Distinguishing first from subsequent submission requires state that varies across submissions, which V by definition lacks.

Witnessed completeness. Fix a witness W and consider two executions: E_1 serving n actions, and E_2 serving the same first $n - 1$ and then a further action whose record the adversary omits. Let d_1, d_2 be the honestly-signed documents. Every attestation in d_2 is genuine: the collector signs truthfully over what it recorded, and each retained receipt is authentic. So d_1 and d_2 are identically distributed over every field V can inspect, and $V(d_1) = V(d_2)$. Since V must accept d_1 (a correct execution), it accepts d_2 . The only field that could differ is a commitment to W ’s served count, which W alone can produce — and requiring it makes the verifier’s input depend on a trust domain outside d , contradicting statelessness. $\square$

Theorem 2 is why we describe in-toto, SLSA, TUF, AAS-1 and **agent-decision** as record formats rather than as weaker gates. Their verification is by construction a function of the submitted document; the gap is structural, not a matter of maturity, and Section 7.6 measures it.

3 System Architecture and Threat Model

Intuition. EviAssure turns an agent’s step-by-step execution path into a signed, Merkle-committed evidence bundle before deployment, and evaluates that bundle at a *fail-closed gate* that denies release whenever a required

proof, signature, or threshold is missing, malformed, or compromised.

3.1 System Model

The EviAssure release assurance architecture involves four primary entities: the *Evidence Collector*, the *Key Management System (KMS)*, the *Policy Registry*, and the *Release Assurance Gate* (Figure 1).

Evidence Collector: in production, an out-of-process monitor outside the agent’s sandbox, observing activity via eBPF syscall tracepoints and container network proxies. *In this evaluation it is Python instrumentation wrappers* — the paper’s principal validity threat (Section 7.10). **Cloud KMS / HSM:** signing authority is rooted outside the runner; the collector obtains signatures over canonical digests without holding key material. The gate checks the declared ARN against `kms_key_arn_pattern`, a misconfiguration control rather than an adversarial one (Section 7.10). **Policy Registry:** an out-of-band store for release policies, pinned public keys, and the revocation list. **Release Assurance Gate:** the enforcement entity, running in a trusted CI/CD step, evaluating the signed `EvidenceBundle` against the pinned policy.

3.2 Adversary Model

The security model considers a powerful adversary $\mathcal{A}$ attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [7, 25, 26]. The adversary $\mathcal{A}$ may control intermediate runner environments, network transit layers, or build scripts.

Trust Model Boundaries. EviAssure assumes Ed25519 and SHA-256 remain computationally intractable for PPT adversaries, and that the Cloud KMS / HSM faithfully enforces identity access control over private signing keys. It further requires strict environment isolation: an agent that escapes its sandboxed tier to obtain local file access or code execution on the assurance engine or the collector host can forge an evidence payload *prior* to KMS signing. Compromise of the assurance engine’s trusted computing base is therefore an explicit out-of-scope limitation — the cryptographic guarantees hold assuming a tamper-proof control plane. HSM side channels, subversion of provider root keys, and hypervisor rootkits are likewise out of scope. *Witness* keys are a distinct trust domain: Section 4.2 assumes the adversary holds the agent and collector keys but no witness key, and Section 7.10 states what degrades when that assumption is weakened.

We categorize $\mathcal{A}$ ’s in-scope capabilities into five classes, mapped to vectors in Table 1: runner and memory tam-

pering ($\mathcal{A}_1$), network and signature forgery ($\mathcal{A}_2$), replay and time skew ($\mathcal{A}_3$), governance bypass ($\mathcal{A}_4$), and serialization malleability ($\mathcal{A}_5$).

3.3 Security Requirements

The construction is proved against four requirements, subject to the trust boundaries above. The evaluation in Section 7 is organised to test each in turn.

1. **Evidence Integrity:** no recorded action can be modified, reordered, inserted, or removed without detection (Definition 3).
2. **Trace Completeness:** the attestation must bind the count of recorded actions so that truncation is not silent (Definition 4) — and, because a count the collector itself signs cannot constrain the collector, must be strengthened to hold *relative to a witness set* (Definition 5).
3. **Release Authorization Soundness:** the decision must be a deterministic function of signed evidence and pinned policy. No PPT adversary should induce an APPROVED verdict for modified or malicious traces.
4. **Trace Privacy (Parameter Blinding):** sensitive parameters and intermediate data must reach public verifiers and auditors only as salted HMAC digests.

The gate enforces these fail-closed: any payload failing verification is rejected with exit code 1 (BLOCKED). Requirement 2 is where every prior system stops at the *collection boundary* [1, 14]; Section 4.2 is our attempt to move it.

4 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser Document Object Model (DOM) attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

4.1 Execution Trace & Browser UI Attestation Schema

Each execution trace record T_i captures structured action metadata across autonomous agent steps. For API and backend tool calls, T_i records:

$$T_i = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{status}_i, \text{duration}_i, H_{\text{out},i}) \quad (2)$$

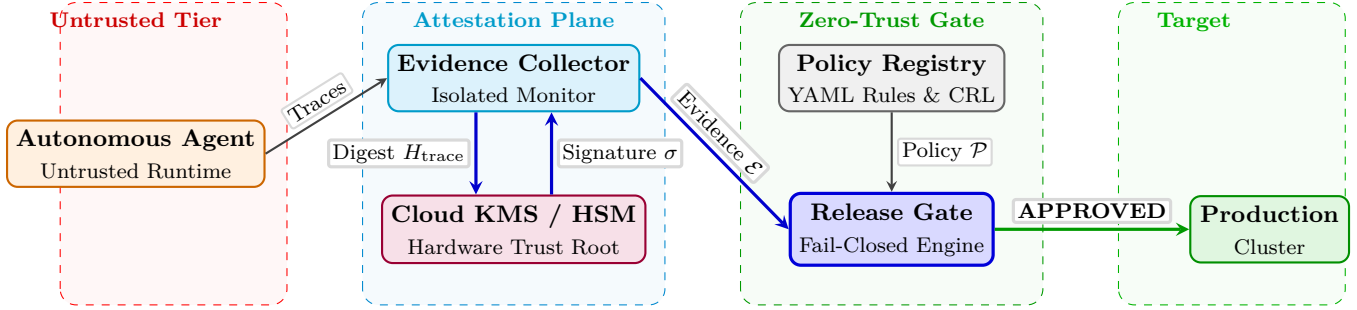


Figure 1: EviAssure System Architecture and Trust Boundaries. The Evidence Collector captures execution traces across an out-of-process monitor boundary and requests hardware-rooted attestation from Cloud KMS. The Release Assurance Gate evaluates the signed bundle ($\mathcal{E}$) against pinned governance rules ($\mathcal{P}$) from the Registry to authorize production release.

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes (H_{DOM}) and visual screenshot render digests (H_{screen}):

$$T_{\text{UI},i} = (\text{id}_i, \text{agent}_i, \text{action}_i, \text{url}_i, \text{elem}_i, H_{\text{DOM},i}, H_{\text{screen},i}) \quad (3)$$

$H_{\text{DOM},i} = \text{SHA256}(\text{DOM}_{\text{serialized}})$ is computed over the serialized DOM of the rendered page: the specimen serializes deterministically before hashing, so identical page states yield identical digests while any mutation — an injected modal dialog, an altered form control — changes the digest. Render digests $H_{\text{screen},i}$ capture canvas pixels under deterministic software rasterization — headless Chromium, GPU and font hinting disabled, pinned viewport, immutable base image — which eliminates the subpixel drift that would otherwise make an honest re-render fail its own digest. Each record yields a canonical leaf digest $H_i = \text{SHA256}(T_i.\text{to_hash}())$ committing *every* schema field — identifier, agent, action, status, duration and output digest — so editing any recorded field after signing, including the timing `duration_ms`, invalidates the recomputed root and is rejected by the gate.

4.2 Witness Protocol

A witness is the component that *actually serves* an agent action: a tool endpoint, an API gateway, a database proxy, a sandbox syscall broker. It is not part of the agent runtime, and the security argument rests on one property: the adversary who owns the agent and the collector does not hold witness keys.

Each witness keeps a monotonic counter per session. On serving action a in session sid it returns

$$r = \text{Sign}_{W_j}(\text{sid}, \text{seq}, H_{\text{act}}(a)) \quad (4)$$

where H_{act} digests the action identity the witness can see (requester, operation, returned digest) rather than

the full Merkle leaf, whose timing fields are collector-side. At session end W_j emits $\text{Sign}_{W_j}(\text{sid}, n_j)$, committing to how many actions it served. **Every registered witness closes every session**, including with $n_j = 0$: without that rule, dropping a witness’s actions *and* its closing is indistinguishable from never using the tool, which is vector O3.

The gate reconciles: authenticate every attestation under a registry-pinned witness key; require the session identifier to match; require each witness’s receipts to form exactly $1..n_j$; require every receipt to bind to a submitted trace; and require every trace claiming a mediated action to carry a receipt. The bundle signature covers a digest of the whole attestation set, so a collector cannot delete receipts from a signed bundle and leave the signature intact.

Cost and deployment. A receipt is one Ed25519 signature over 100 bytes at the point of service, and the gate performs $O(|T|)$ verifications. The real cost is organisational, not computational: witnesses must be deployed at the boundaries that matter, and Definition 5 makes the resulting guarantee exactly as strong as that deployment. We regard stating the guarantee relative to a declared, auditable witness set as an improvement over assuming a perfect collector, not as a weaker claim.

Parameter blinding. Bundles carry digests, never raw payloads. A keyed HMAC mode (`blind_payload`) additionally re-keys those digests per deployment, which buys unlinkability across environments and resistance to dictionary recovery of low-entropy outputs — $\text{SHA256}(\text{"TOKEN_VALIDATED"})$ is recoverable by exhaustive search, the salted HMAC is not. It is deterministic, so it does not conceal repetition within one deployment. We record it as a hardening measure rather than a contribution.

4.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest N execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [17, 19]. Each trace record T_i maps to leaf digest H_i .

For $N = 1,000$ traces, transmitting raw trace logs requires 222.6 KB of storage, scaling to 220,705.03 KB for $N = 1,000,000$ traces. EviAssure provides sparse Merkle proof compression, transmitting the 32-byte root R_M and $O(\log N)$ inclusion proof paths π_i for audited steps.

Given proof path $\pi_i = \{(P_1, \text{pos}_1), \dots, (P_K, \text{pos}_K)\}$, inclusion is verified recursively by setting $C_0 = H_i$ (the committed leaf digest; leaf digests are already SHA-256 strings and are *not* re-hashed at the proof boundary, matching `verify_merkle_proof`) and evaluating:

$$C_j = \begin{cases} \text{SHA256}(C_{j-1} \parallel \text{"."} \parallel P_j), & \text{if } \text{pos}_j = \text{right} \\ \text{SHA256}(P_j \parallel \text{"."} \parallel C_{j-1}), & \text{if } \text{pos}_j = \text{left} \end{cases} \quad (5)$$

Leaves and internal nodes are hashed under *disjoint domain prefixes* following RFC 6962 [17]: a leaf commits $\text{SHA256}(0x00 \parallel T_i.\text{to_hash}())$ and an internal node commits $\text{SHA256}(0x01 \parallel L \parallel R)$ over the raw 32-byte child digests. An earlier revision of this system instead hashed both as unprefixed SHA-256 over an ASCII string with a `:` delimiter and argued that the fixed 129-byte node preimage supplied domain separation. It does not: the two input languages merely failed to overlap for the leaf schema in use, which is an accident of the schema rather than a property of the construction. Explicit prefixes make the separation structural.

The proof is valid if and only if $C_K = R_M$ and the path length equals the committed tree depth $d = \lceil \log_2 N \rceil$ derived from the signed execution trace count. The two defences are independent and both are necessary: domain separation prevents an internal digest from ever being interpreted as a leaf, and the depth bound prevents the shortened-path substitution of the CVE-2012-2459 family, which succeeds without any hash collision. Vector V15 (Section 7.4) exercises exactly this attack against the third-party auditor path; earlier revisions of this system failed it. Transmission size for an audited step remains < 5 KB.

```
LEAF, NODE = b"\x00", b"\x01" # RFC 6962 domain
separation

def leaf_digest(content):
    return sha256(LEAF + content.encode()).hexdigest()

def node_digest(l, r):
    return sha256(NODE + bytes.fromhex(l)
                  + bytes.fromhex(r)).hexdigest()

def build_merkle_tree(leaf_digests):
    current, levels = leaf_digests, [leaf_digests]
    while len(current) > 1:
```

```
        current = [node_digest(current[i],
                                current[i+1] if i+1 < len(current)
                                else current[i])
                    for i in range(0, len(current), 2)]
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

Listing 1 implements the recursive aggregation of agent trace hashes into the final $O(\log N)$ verifiable structure.

4.4 SLSA v1.0 and in-toto Predicate Envelopes

To integrate with cloud-native artifact registries and software supply chain attestors, EviAssure formats each generated `EvidenceBundle` into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [27, 29]. The envelope carries `_type` `https://in-toto.io/Statement/v0.1`, the artifact digest as `subject`, `predicateType` `https://slsa.dev/provenance/v1`, and the `evidence_id` under `predicate.buildDefinition.externalParameters`; a complete example is in the artifact.

The envelope binds the target release artifact — an agent weight package, code container, or configuration tarball — to the evidence attestation. `subject` carries the artifact filename and SHA-256 digest; `predicateType` asserts SLSA Provenance v1.0; within `predicate.buildDefinition`, `buildType` names the gate evaluation parameters and `externalParameters` anchors the `evidence_id`, binding the evidence pack to the SLSA statement.

5 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (`ReleasePolicyEngine`) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and the system-level formal security analysis.

5.1 Declarative Policy Governance Rules

A release policy specification P defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\min_pass}, P_{\max_drift}, P_{\min_sigs}, P_{\text{valid_window}}, \mathcal{K}_{\text{trusted}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \quad (6)$$

where:

- $P_{\min_pass}$: Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\max_drift}$: Maximum allowable unresolved security drift findings (default: 0).
- $P_{\min_sigs}$: Required threshold number of valid signatures from authorized keys (default: $K = 1$).
- $P_{\text{valid_window}}$: Evidence timestamp freshness validity window $\Delta T_{\max} = 3,600$ s with future clock-skew tolerance $\delta_{\text{skew}} = 30$ s (assuming RFC 5905 NTP synchronisation).
- $\mathcal{K}_{\text{trusted}}$: Trusted key registry pinning authorized key IDs to Ed25519 public keys (`governance/trusted_keys.yaml`). Verification uses the pinned key, never one supplied inside a bundle; unregistered IDs are rejected. Authority thus rests on a small, versioned, offline-pinnable root set, following key transparency [18] and delegation hierarchies [15].
- $\mathcal{K}_{\text{rev}}$: Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$: Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 2 outlines the complete zero-trust verification algorithm enforced by `ReleasePolicyEngine`.

```
# In: Evidence E, Policy P, revoked K_rev, seen N_seen
# Out: D in {APPROVED, BLOCKED}; any exception =>
      BLOCKED
def evaluate_release_gate(E, P, K_rev, N_seen):
    if N_seen is None: return BLOCKED # no replay
    state
    valid = set()
    for s in E.signatures:
        if s.key_id in K_rev: continue
        if s.key_id not in K_trusted: continue #
            registry
        pk = K_trusted[s.key_id] #
            pinned only:
        if s.pub_key and s.pub_key != pk: continue
        if not match_kms_arn(s.kms_arn, P.kms):
            continue
        if verify_ed25519(canon(E), s.sig, pk):
            valid.add(s.key_id) # distinct
    signers
    if len(valid) < P.min_signatures: return
        BLOCKED
    if merkle_root(E.traces) != E.merkle_root: return
        BLOCKED
    if E.traces_count != len(E.traces): return
        BLOCKED
    if E.test_pass_pct < P.min_pass_pct: return
        BLOCKED
    if E.unresolved_drift > P.max_drift: return
        BLOCKED
    if E.nonce in N_seen: return
        BLOCKED
    if not fresh(E.timestamp, P.window, skew=30):
        return BLOCKED
    N_seen.add(E.nonce); return APPROVED
```

Listing 2: Fail-Closed Policy Gate Verification Protocol.

Listing 2 outlines the zero-trust policy engine logic that strictly enforces fail-closed behavior on signature or trace discrepancies.

5.2 Formal Security Analysis

The analysis has two layers. Theorem 3 is primitive-level: a game sequence reducing to EUF-CMA and collision resistance. Theorem 4 is system-level: what an APPROVED verdict *means*, and the trust boundaries — collector, key registry, nonce state — that meaning rests on.

Theorem 3 (Fail-Closed Release Soundness). Under the assumption that Ed25519 [3] is EUF-CMA secure [5, 9] and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary $\mathcal{A}$ can induce `ReleasePolicyEngine` to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability $\text{negl}(\lambda)$.

Proof. Two hops carry content. **Game₁** aborts if $\mathcal{A}$ produces a valid signature over a payload the honest packager never issued; an EUF-CMA adversary $\mathcal{B}$ installs its challenge key as the sole registry entry and answers packaging requests through its signing oracle on $\text{canon}(E)$, so the simulation is perfect and a **Game₁** abort is a forgery on a never-queried message. **Game₂** aborts on $\mathcal{T}^* \neq \mathcal{T}$ with equal root; under domain-separated leaves and the committed depth this needs a SHA-256 collision. Replay, freshness, revocation, registry membership and threshold comparison are table lookups over the signed payload, rejecting with probability 1 given correct gate state, and contribute no term. Hence $\text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{B}, t, q_s) + \text{Adv}_{\text{SHA256}}^{\text{coll}}(\mathcal{C}, t)$, read at the 128-bit level rather than asymptotically in λ , since both primitives are fixed-parameter. $\square$

Theorem 3 bounds an adversary’s ability to *fake* evidence; it says nothing on its own about what a genuine, honestly-signed evidence pack entitles its holder to. That is the release-gate question, and it is Theorem 4.

Theorem 4 (Evidence-Bound Release Authorization Soundness). Fix a policy version P and a trusted key registry $\mathcal{K}_{\text{trusted}}$. If `ReleasePolicyEngine` outputs APPROVED for evidence E , then, except with probability $\text{negl}(\lambda)$, all of the following hold:

- (i) **Authenticity:** E carries a valid Ed25519 signature by a non-revoked key pinned in $\mathcal{K}_{\text{trusted}}$; attacker-generated keys, spoofed key IDs, and revoked keys cannot produce a valid signature.
- (ii) **Evidence binding:** the recomputed Merkle root over the submitted trace set equals the signed root, the signed trace count equals the number of submitted traces, and every inclusion proof binds to the committed tree depth under disjoint leaf/node domains. The trace set the gate evaluated is therefore

the one the collector signed — no trace was added, removed, reordered or edited between collection and adjudication. This says nothing about the relation between the signed trace set and the underlying execution; that gap is Definition 4 and is not closed by any cryptographic check.

- (iii) **Freshness:** the timestamp lies within the validity window $\Delta T_{\max}$ and the skew bound δ_{skew} , and the nonce has not been processed within the window; the release is not a replay of an earlier approved bundle.
- (iv) **Policy compliance:** all declared thresholds (test pass rate, unresolved drift, KMS ARN boundary, signature threshold K) satisfy policy version P ; an APPROVED verdict under a different policy version is not an APPROVED verdict for this release.

Scope and Operational Assumptions. The guarantees hold conditionally on (a) the *collector assumption* of Definition 4 — the evidence collector observed and recorded every action of the release execution — which supplies the completeness half of the guarantee, and (b) the *key-management assumption* — registry keys are honestly generated and the HSM/KMS enforces the ARN boundary. The theorem asserts nothing about actions that never reached the collector, nor about the semantic correctness of recorded trace content; both are defined in the threat model and handled by the out-of-band semantic inspection layer.

Proof. (i) Follows from Theorem 3, Game_1 , plus the deterministic registry checks enforced before any signature verification (`policy.py`): unknown key IDs, bundle-supplied keys disagreeing with the registry, and revoked keys are rejected outright, so per-signature success is bounded by $\text{Adv}_{\text{Ed25519}}^{\text{euf-cma}}(\mathcal{A})$. (ii) Follows from Theorem 3, Game_2 , plus the deterministic signed-count check $N = |\text{traces}|$; substitution requires either a hash collision or a broken signature. (iii) Freshness, skew, and nonce checks are deterministic (Game_3 ; Listing 2) and hold with probability 1 given correct gate state, with the disclosed caveat that replay protection requires sharing the nonce cache across gate replicas in distributed environments. (iv) Threshold comparisons are deterministic over the signed payload and the pinned policy version. Composition yields the statement; the only probabilistic terms reduce to the primitive advantages of Theorem 3. $\square$

6 Deployment Integration and Workload Specimens

EviAssure runs as a GitHub Action against two synthetic workload specimens: a backend multi-step agent

(`specimens/agent_runner.py`) exercising JWT claims, vector retrieval and sandboxed SQL, and a browser agent (`specimens/web_app_runner.py`) exercising navigation, clicks, DOM-state hashing and screenshot digests. Both are simulations. A third specimen (`specimens/live_agent_runner.py`) is not: it drives a real tool-calling LLM whose tools are fronted by witnesses, and Section 7.9 reports what that produced. The production collector of Section 3 (eBPF syscall mediation) remains an architectural proposal rather than an evaluated component — which is precisely why Section 4.2 moves the completeness guarantee onto witnesses that a compromised collector cannot speak for.

6.1 CI/CD Integration

The gate ships as a GitHub Action that loads the pinned key and witness registries and the policy, ingests the `EvidenceBundle`, runs `ReleasePolicyEngine` (`scripts/verify_release_gate.py`), and on failure emits structured diagnostics and exits non-zero on every output path.

6.2 Agent Execution Trace Corpus ($N = 1,075$)

The corpus (`corpus/agent_trace_corpus.json`) contains 1,075 profiles / 5,450 trace records across five agent architectures, each profile carrying four to six steps: 1,000 clean, 50 *overt* anomalies over ten classes, and 25 *stealth* anomalies over five classes. It is entirely synthetic and templated; this is stated in the artifact’s own **provenance** field and is the principal threat to external validity (Section 7.10).

The overt/stealth split exists because an earlier revision of this corpus was not a benchmark. Every planted anomaly used an action name and status absent from clean traffic, and the detector keyed off a hard-coded list of exactly those names: generator and detector were one list, written together, so the reported recall was a property of the generator. *Overt* classes retain that structure (out-of-vocabulary action and status) and are the easy half. *Stealth* classes are the point: a mutating statement issued through the read-only query path, a ledger transfer to an attacker-controlled destination, retrieval returning injected context, extraction used as an exfiltration channel, a linter invoked with findings suppressed. Each uses an action, a status and a duration drawn from clean traffic, so no per-leaf vocabulary or envelope test can separate them. Section 7.8 reports both halves separately.

6.3 Forensics and Diagnostics: CLI Inspection & Audit Tool

To support incident response after a release has already occurred, EviAssure ships an offline forensic inspection utility (`scripts/audit_trace_forensics.py`). It parses serialized `EvidenceBundle` files, reconstructs the full Merkle tree, verifies Ed25519 signatures against the pinned registry and revocation list, and isolates leaf-index anomalies. Presented with a tampered payload it reports the exact leaf position i and the output-hash mismatch, emitting structured JSON for security operations centre (SOC) ingestion.

7 Systems Security Evaluation

The empirical evaluation follows a deliberate two-part strategy: a micro-level validation of the gate’s enforcement mechanisms, and a macro-level evaluation of the system deployed at corpus scale.

Mechanism evaluation. Baseline performance is measured through Merkle scaling latency and multi-core verifier throughput. Adversarial resilience is measured against the vector suite: twelve vectors derived from the five capability classes ($\mathcal{A}_1$ – $\mathcal{A}_5$), each exercising one enforcement, plus five authored against the design rather than against the check list — the ones with the power to fail, and three of which did. Clean negative controls measure the false-block rate, without which a block rate is uninterpretable.

System-level deployment. A corpus-scale evaluation over 1,075 agent trace profiles exercises the interaction between the cryptographic integrity gate and an out-of-band semantic inspection layer — and, importantly, shows where that interaction still leaves a gap.

Results are split by kind. *Deterministic* security results (block counts, ablation, inspection) are produced by `scripts/run_security_eval.py` into `results/security_evaluation.json` and are platform-independent: they reproduce identically on any machine. *Timed* measurements are wall-clock timings recorded by `scripts/run_release_benchmark.py` into `results/benchmark_summary.json` on an Apple M4 Max with 14 cores (10 performance, 4 efficiency; no SMT, so logical and physical core counts coincide) running Python 3.14.x. Each scaling point, throughput configuration, proof cost, blinding cost, and hashing figure is the mean over 5 runs, with the standard deviation recorded alongside in the artifact (`*_std` fields); per-worker throughput aggregates 1,000 policy evaluations per run. Tamper vectors and gate verdicts are deterministic checks evaluated once each, so run-to-run dispersion there is zero by construction; sampling uncertainty over the finite vector set is reported instead, as a Wilson 95%

interval on every rate.

7.1 Merkle Scaling up to $N = 1,000,000$ Traces

Merkle build time was evaluated from $N = 10$ to $N = 10^6$ over 5 runs per point. Construction scales to 10^6 traces in 378.00 ms (mean; $\sigma = 6.21$ ms across runs), and to 37.3 ms for $N = 100,000$. We previously reported a “packaging overhead” percentage here and have withdrawn it. The numerator timed only post-tree bundle construction and signing — a constant, independent of N — while the denominator grew as $1.5 \text{ ms} \times N$, so the ratio fell monotonically for arithmetic reasons rather than measuring any property of the system. Merkle construction, the component that actually scales, is reported directly above.

7.2 Parallel Verifier Throughput

Verifier throughput was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests per run and 5 runs per configuration; each evaluation is a distinct fresh-nonce bundle against a warm policy engine (evidence creation and replay-state checks are measured separately, so the timed region is steady-state verification). Throughput rises from 3,728 ops/s on a single worker to a peak of $7,152 \pm 179$ operations/second (mean $\pm$ std) at 4 workers (a $1.9\times$ speedup), then declines. We do not attribute the decline to a hardware effect: each task is a three-trace bundle shipped to a worker process, so the measurement is dominated by inter-process serialization rather than verification. Two consequences must be read together with Section 7: this figure characterises *small* bundles, and because the gate recomputes the whole tree (Listing 2, step 2), per-bundle verification cost is linear in N — at $N = 10^6$ the gate is bounded by the 378.00 ms rebuild, not by this throughput number.

7.3 Sparse Inclusion Proofs

Auditors verifying individual high-risk steps should not re-process full logs (220,705 KB at $N = 10^6$). At $N = 10^6$ the committed depth is $d = \lceil \log_2 N \rceil = 20$; proof generation takes 0.003 ms, yielding a 1.93 KB payload of 20 node digests that verifies in 0.007 ms — a bandwidth reduction of roughly 99.999%.

7.4 Adversarial Vector Suite

EviAssure blocks 16 of 17 scored vectors (94.1%, 95% CI [73.0, 99.0]), and falsely blocks 0 of 4 clean negative

Table 1: Adversarial vectors and gate outcomes. V1–V12 target one enforcement each; V14–V18 were authored against the design, independently of the check list. V13 is retained as a configuration check but is no longer scored (Section 7.10).

ID	Vector	Class	Outcome
<i>Mechanism vectors (one per enforcement)</i>			
V1	Unsigned payload	$\mathcal{A}_2$	Blocked
V2	Tampered trace digest	$\mathcal{A}_1$	Blocked
V3	Attacker-key signature	$\mathcal{A}_2$	Blocked
V4	Replayed nonce (same gate)	$\mathcal{A}_3$	Blocked
V5	Expired timestamp	$\mathcal{A}_3$	Blocked
V6	Unresolved drift	policy	Blocked
V7	Sub-threshold pass rate	policy	Blocked
V8	Forged Merkle root	$\mathcal{A}_1$	Blocked
V9	Revoked key ID	$\mathcal{A}_4$	Blocked
V10	Duplicate-key wire doc	$\mathcal{A}_5$	Blocked
V11	Future clock skew	$\mathcal{A}_3$	Blocked
V12	Trace-set truncation	$\mathcal{A}_1$	Blocked
<i>Independent vectors (authored against the design)</i>			
V14	Attacker key, registry off	$\mathcal{A}_2$	Blocked [†]
V15	Internal node as leaf	$\mathcal{A}_1$	Blocked [†]
V16	Cross-replica nonce replay	$\mathcal{A}_3$	Not blocked
V17	64-char leaf confusion	$\mathcal{A}_1$	Blocked [†]
V18	Post-signing DOM mutation	$\mathcal{A}_1$	Blocked
Scored total		16/17 (94.1%)	
Clean negative controls		0/4 falsely blocked	

controls. Three points about how that number should be read.

The mechanism vectors do not demonstrate resilience. V1–V12 map close to one-to-one onto the gate’s checks, because we wrote both; they are a completeness test for the implementation and nothing more. An earlier version argued this 1:1 map proved complete coverage; V14 and V17 are why that is invalid.

Three vectors (†) broke earlier revisions. V14 exploits a disabled trusted-key registry: the gate verified against the key carried *inside the bundle*, so an adversary with their own keypair and a conformant ARN was APPROVED with zero violations. V17 exploits a `len(x) != 64` heuristic, letting a 64-character leaf string be adopted as its own digest. V15 is the CVE-2012-2459-family internal-node substitution against the auditor path. None was reachable by the mechanism suite, which is the argument for authoring vectors against the design.

V16 is not blocked, and we report it. The same approved bundle presented to a second gate replica that does not share nonce state is APPROVED, because replay protection is a property of gate *state*, not of the evidence. The shipped verifier persists nonces and fails

closed when the ledger is unavailable, but cross-replica correctness needs shared storage the prototype does not provide.

Differential wire fuzzing. The previous campaign reported 1,000/1,000 blocked and carried no information: every mutation perturbed a field inside the signed payload, so a verifying bundle was unreachable by construction, and it contained no clean cases. The replacement perturbs the *serialization* of a validly signed bundle — key order, insignificant whitespace, unicode escaping, number formatting, duplicate keys, byte-order marks — where both outcomes are reachable. Over 300 cases (20% clean controls): 52/52 semantics-changing encodings blocked, 248/248 semantics-preserving encodings approved. This campaign found a real defect: the duplicate-key detector accumulated key counts across every object in the document, so sibling objects in the `traces` array registered as duplicates and any deployment actually supplying the raw wire text rejected every multi-trace bundle. It is fixed and regression-tested.

7.5 Per-Check Ablation

Each enforcement was disabled in turn and the suite re-evaluated (matrix in the artifact’s `ablation` block). Disabling the signature check releases V1, V3, V9, V14 and V18; removing Merkle re-derivation releases V2 and V8; the quality, drift, freshness and replay checks each release exactly the vectors they target (V7; V6; V5 and V11; V4). V16 escapes every variant, including the full gate.

Two results are worth stating precisely, because the previous version of this section stated one of them incorrectly.

First, **not every check is necessary against this suite**. Disabling the signed trace-count binding releases no vector: V12 is independently caught by root recomputation. The count binding earns its place by protecting the *auditor* path — proof depth is derived from the signed count — not by adding coverage at the gate. We previously wrote that “every check is necessary”; the artifact’s own matrix contradicted it.

Second, **emptying the trusted-key registry collapses the gate rather than weakening it**: all Ed25519 evidence is rejected because no signature can be authenticated. This is now true of the implementation. When we first claimed it, the ablation flag redirected verification to the bundle-supplied key rather than disabling it — the V14 bypass — so establishing the claim required fixing the code, not re-reading the matrix.

Table 2: Omission attacks detected, out of 6. All systems approve the honest control (OC1). Baselines are executed, not modelled.

	Verification approach	Detected
in-toto/DSSE	signature over the document	0/6
TUF	expiry, thresholds, revocation	0/6
AAS-1 style	per-issuer hash chaining	1/6
Receipts (2026)	per-action receiver attestation	2/6
EviAssure	full gate, no reconciliation	0/6
EviAssure	+ witnessed completeness	6/6

7.6 Omission Attacks

The tamper suite and the omission suite test different things. A tamper vector modifies evidence and is caught by an integrity check. An omission vector modifies *nothing*: the adversary performs an action and declines to record it, then signs honestly over what remains. There is no discrepancy to detect, which is why Theorem 2 rules the class out for document verifiers rather than merely predicting they will do badly at it.

We construct six (Table 2) plus an honest fully-witnessed control that every system must approve — without which a reconciler that rejects everything would score perfectly.

Three results are worth reading carefully. First, **EviAssure without reconciliation detects 0 of 6** despite enforcing signatures, Merkle re-derivation, count binding, freshness and replay. The mechanism, not some incidental check, is what closes the class.

Second, **per-action receipts detect 2 of 6**. Receipts catch the two attacks that leave a trace *without* an attestation — cross-session splicing and fabrication — and miss all four truncations, because dropping an action drops its receipt and the remaining pairs still match. This is the empirical form of the paper’s central observation: a receipt proves an action happened; a set of receipts does not prove no other action happened.

Third, **hash chaining detects 1 of 6**. A chain links records that are present, so an interior gap breaks it and a suffix truncation does not — the end simply moves. This is the precise limitation of AAS-1’s omission story, and it is why the closing count, not the chain, is what makes truncation detectable.

7.7 Comparative Gate Analysis

The previous version of this comparison was not defensible. It scored strawman baselines — a “Sigstore/Cosign-style” gate that was a presence check on a JSON field, an “exit-code gate” that was `test_pass_pct > 0` — against

a vector set derived from our own check list, and the resulting near-zero figures reached the abstract as though they characterised the state of the art. They did not. We have rebuilt it.

Each baseline is now configured as a competent engineer would deploy it, and executed:

1. **in-toto/DSSE verification** (executed, `securesystemslib DSSE PAE + Ed25519`): authenticates the envelope against a *pinned trust root* rather than a key travelling with the payload. Blocks 6/17 (35.3%, CI [17.3, 58.7]).
2. **TUF metadata verification** (executed, `python-tuf`): real Root/Timestamp metadata with `expires`, signature thresholds and root-driven revocation. Blocks 8/17 (47.1%, CI [26.2, 69.0]).
3. **OPA Rego policy** (executed when the `opa` binary is present; `governance/baseline_opa.rego`): 3/17 (17.6%).
4. **Composed pipeline** ($\text{DSSE} \wedge \text{TUF} \wedge \text{OPA}$): 10/17 (58.8%, CI [36.0, 78.4]).
5. **Unauthenticated status gate**: 0/17. Retained explicitly as a *floor*, not a competitor.

The honest headline is therefore 16/17 versus 10/17, and the intervals overlap: with 17 vectors this evaluation cannot support a strong quantitative separation, and we do not claim one. What it does support is a *qualitative* account of which vectors survive a modern composed pipeline and why. The composed baseline blocks every authentication, freshness, revocation and policy-threshold vector; TUF alone blocks 8 (Section 8.3).

The vectors it does not block are the ones this paper is actually about: V2 and V8 (trace-level integrity — no baseline recomputes a Merkle root over an action history), V12 (trace-set truncation), V15 and V17 (proof-path and leaf-typing attacks on the audit surface), and V4 (within-gate replay, which requires the verifier to hold state across submissions rather than to validate a document). That is the delta: *binding the release decision to the execution trace, and holding state while deciding*. It is a narrower claim than 100% versus 30.8%, and it is one the evaluation can carry.

7.8 Corpus-Scale Layered Evaluation

Neither the tamper nor the omission suite reaches the semantic axis: a complete, witnessed, correctly signed bundle can still record behaviour nobody authorised. The corpus measures that third layer (Table 3).

L1 and L2 are as before: all bundles are Merkle-consistent with valid depth-bound proofs, all clean profiles are APPROVED, and *all* anomalous profiles are also

Table 3: Layered evaluation over 1,075 profiles / 5,450 trace records. L3 uses a detector fitted on clean profiles only.

Layer	Check	Result	Rate
L1 Integrity	Merkle root consistency	1,075/1,075	100%
L1 Integrity	Depth-bound proofs valid	1,075/1,075	100%
L2 Gate	Clean APPROVED	1,000/1,000	100%
L2 Gate	Anomalous AP-PROVED	75/75	100%
L3 Inspect	Overt recall	50/50	100%
L3 Inspect	Stealth recall	0/25	0%
L3 Inspect	Overall recall	50/75	66.7%
L3 Inspect	Clean falsely flagged	0/1000	0%

APPROVED. We report the latter rather than hiding it — semantic misbehaviour that is cryptographically well-formed is outside the gate’s scope by design.

L3 is the layer whose evaluation we rebuilt. The detector no longer consults any list of anomalous actions. It is fitted on clean profiles alone, learning an action vocabulary, a status vocabulary and a per-action duration envelope, and every anomaly class is held out from that fit. Under this protocol it recalls 50/50 overt anomalies and 0/25 stealth anomalies — overall 50/75 (66.7%, CI [55.4, 76.3]) at 0/1000 false positives.

The zero is the result. Per-leaf inspection detects agents that do things they have never done before; it is blind to an agent doing a permitted thing for an impermissible reason, which is precisely what a prompt-injected agent looks like in a trace. Combined with L2, the layered picture is: the gate proves what was recorded is genuine, inspection catches structural novelty, and *neither* catches semantically disguised misbehaviour. The earlier 50/50 recall reported here was a property of that shared list (Section 6.2), not a measurement.

7.9 Live Agent Sessions

Everything above is computed over a templated corpus, which leaves the paper’s own premise untested: does witnessed completeness survive an agent that actually decides what to do? We therefore ran 14 sessions of a real tool-calling LLM (`openai/gpt-oss-120b`) whose tools are fronted by the witnesses of Section 4.2. The model chooses the actions; nothing is scripted.

The sequences are genuinely non-deterministic — 10 of 14 sessions were distinct (71%), over 7.36 ± 1.646 actions — and the agent generated its own failures, malformed SQL refused by the read-only path (`DENIED`) and queries against non-existent tables (`ERROR`), rather than executing a happy path we wrote for it.

Reconciliation is sound on real traces: the honest control reconciles 14/14, and re-deriving the omission

attacks per session detects 69/69. Detection is cryptographic, so this confirms rather than surprises — but a protocol that failed here would have been refuted.

Witness coverage was 93.2% on average, 66.7% at worst. This is the quantity Definition 5 leaves open: the guarantee holds relative to the witness set, and that set covered most but not all of what a real agent did. The residue was `emit_summary`, a local bookkeeping call no witness fronts by construction. That is the honest shape of the property — completeness over the actions a deployment chose to mediate, with a measurable remainder rather than a hypothetical one. 1 of 15 sessions was lost to a provider-side output-parsing error, which we report rather than retry into silence.

7.10 Limitations and Threats to Validity

The corpus is still synthetic. Section 7.9 runs a real tool-calling LLM against the witness protocol, so the completeness claim is no longer evaluated only on templates. The 1,075-profile corpus behind Sections 7.8 and the semantic-inspection results is not: it remains synthetic and templated,, as the artifact’s own **provenance** field declares. The layer this paper contributes is now exercised on non-deterministic behaviour; the layer it inherits is not. The live sessions are also modest in scale (14), single-provider, and use benign tools; they establish that the protocol composes with a real agent and what coverage it achieved, not how it behaves under an adversarial one.

The collector is unimplemented. Every security guarantee is conditioned on complete mediation at the collection boundary (Definition 4). The production design grounds this at the OS boundary via eBPF; the evaluation uses Python instrumentation wrappers. The load-bearing component is therefore an architectural proposal, not a measured one.

The gate does not block semantic misbehaviour, and inspection misses the hard half. L2 approves all 75 anomalous profiles by design; held-out L3 inspection recalls 0/25 stealth anomalies (Section 7.8). Content-level policy over trace semantics is future work.

V16 is unblocked. Cross-replica replay protection requires shared nonce storage the prototype does not implement (Section 7.4).

Witness trust and availability. Witnessed completeness is exactly as strong as the declared witness set. An adversary holding a witness key mints valid receipts and closings for that witness, and action types no witness mediates lie outside the guarantee by construction (Definition 5). We evaluate neither witness compromise nor key rotation, and a witness that fails to close a session blocks the release — fail-closed, but a denial-of-service surface a deployment must plan for.

Vector-set construct validity. V1–V12 were authored alongside the checks they exercise and demonstrate only that those checks fire. V14–V18 were authored against the design and three of them broke earlier revisions, which is evidence that the suite has some power — but a 17-vector suite gives wide intervals (CI [73.0, 99.0]) and no suite authored by the system’s designers can establish resilience to novel attack.

Measurement and key management. Timed figures are means over 5 runs on one platform; σ captures within-run noise only. The artifact signs with a static demo key so results reproduce; there is no real KMS round trip, and the 15–25 ms figure is an estimate.

8 Related Work

Positioning. We state the boundary of the claim first: threshold signing, metadata expiry, rollback protection and key revocation are *not* novel here — TUF has supplied them since 2010 [23], and our executed TUF baseline blocks 8 of 17 vectors on that basis alone (Section 7.7). EviAssure bridges a narrower gap than earlier drafts of this paper claimed: it binds the release decision to a Merkle-attested *execution trace* and holds replay state across submissions, which is what none of the systems below does.

8.1 Supply Chain Provenance

in-toto [29], SLSA [27] and Sigstore/Cosign [21] provide cryptographic provenance for build pipelines and image digests, including vetted runtime-trace predicates [12]; any release pipeline should use them for the static artifact side. What they do not provide is the stateful authorization layer of Section 5.2. EviAssure supplies it while remaining envelope-compatible — an **EvidenceBundle** is an in-toto Statement and can carry SLSA provenance, **agent-decision** records, or custom predicates alongside Merkle-attested traces.

8.2 Policy Engines and Workload Identity

Policy engines (OPA [24], Kyverno [28]) evaluate declarative rules over admission payloads but compute no Merkle root over an action history and hold no replay state. Workload identity frameworks (SPIFFE/SPIRE [11], Macaroons [4]) are orthogonal: they establish *who* the runner is, not what action history it produced. Runtime LLM guardrails filter inference-time inputs and outputs; they do not gate the promotion of a checkpoint, prompt template, or tool scope. Update frameworks are the closest prior art on governance: TUF [23] and its delegation extensions [15, 20] supply

expiry, thresholds, rollback and revocation, which is why we execute TUF as a baseline rather than claim those properties.

8.3 AI-Agent Provenance Standards

Three 2026 initiatives address agent provenance as a *record format*. The in-toto **agent-decision/v0.1** RFC [2] standardises the shape of a signed policy-decision record, leaving freshness, replay and revocation to the verifier. APAS [1] builds provenance chains on in-toto Statements and grades itself candidly: at its lower levels provenance is “self-attested — an attacker who compromises the orchestrator can forge records.” AAS-1 [14] defines evidentiary record classes with per-issuer hash chains that detect omission *between* records a verifier already holds — which, as Section 7.6 measures, a suffix truncation evades.

Theorem 2 explains why this is structural rather than a maturity gap: each verifies a submitted document, and no such verifier can achieve replay-freedom or witnessed completeness. Their records ride inside EviAssure’s envelope unmodified; what EviAssure adds is the state and the second trust domain that a document cannot carry.

9 Ethics Considerations

All datasets, trace profiles and harnesses are synthetic and locally generated: no human subjects, no customer PII, no live credentials, and no measurement touching third-party systems. Dual-use considerations — workplace surveillance, parameter blinding, and what this system does *not* prevent — are analysed in Appendix A.

10 Conclusion

EviAssure binds a release decision to a domain-separated, depth-bound Merkle attestation of an agent’s execution trace, evaluated by a stateful fail-closed gate. Its central claim is Witnessed Trace Completeness: reconciling collector evidence against signed witness receipts and closing counts detects 6/6 omission attacks every executed record-format baseline misses, and Theorem 2 shows no stateless document verifier can close that class at all. On the tamper axis it blocks 16/17 against 10 for an executed in-toto/DSSE + TUF + OPA composition, and fails one — cross-replica nonce replay — which we report. Held-out inspection recalls 50/50 overt and 0/25 disguised anomalies: the honest boundary of the contribution. Evidence-bound release assurance makes an unauthorised agent release undeniable and attributable, not impossible.

References

- [1] Agentic Research. Agent provenance attestation standard (apas). Draft v0.2.1, reference implementation “rosary” / `signet`, 2026. Draft standard for cryptographically verifiable provenance chains across autonomous AI-agent pipelines, from work decomposition through agent execution to code delivery; uses in-toto Statement envelopes and DSSE signing.
- [2] Auxidus Technologies. Rfc: agent-decision/v0.1 predicate for ai agent policy decisions. in-toto/attestation issue #554, in-toto Attestation Framework (CNCF), 2026. Open RFC proposing an in-toto attachment predicate for AI agent authorization decisions: agent identity, principal, policy evaluations, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation.
- [3] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [4] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrabie, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.
- [5] Jacqueline Brendel, Cas Cremers, Dennis Jackson, and Mang Zhao. The provable security of ed25519: Theory and practice. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1659–1676, 2021.
- [6] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.
- [7] Nicholas Carlini, Matthew Jagielski, Christopher A Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.
- [8] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In *NDSS Symposium*, 2021.
- [9] Shafi Goldwasser, Silvio Micali, and Ronald L Rivest. A digital signature scheme secure against adaptive chosen-message attacks. In *SIAM Journal on Computing*, volume 17, pages 281–308, 1988.
- [10] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.
- [11] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.
- [12] in-toto Attestation Framework contributors (CNCF). Runtime trace predicate, in-toto attestation framework. Predicate specification, <https://github.com/in-toto/attestation/blob/main/spec/predicates/runtime-trace.md>, 2024.
- [13] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [14] Kadikoy Limited. Agent auditability standard (aas-1), working paper v0.1. Draft for public comment, companion to the AIS-1 agent identity standard, 2026. Open standard for audit-grade evidentiary records of autonomous agent activity: five record classes (Action, Batch, Continuous, Determination, Engagement), twelve audit assertions, JCS canonicalization, SHA-256 hashing, and per-issuer hash chains for omission detection.
- [15] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using delegations to protect community repositories. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.
- [16] Piergiorgio Ladisa, Henrik Plate, Matías Martínez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1509–1526, 2023.
- [17] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2013.

- [18] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium*, pages 383–398, 2015.
- [19] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO’89 Proceedings*, pages 218–238, 1989.
- [20] Marina Moore, Trishank Karthik Kuppusamy, and Justin Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Annual Computer Security Applications Conference (ACSAC)*, pages 1002–1017, 2023.
- [21] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM Conference on Computer and Communications Security (ACM CCS), Demo Track*, 2022.
- [22] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 1811–1825, 2022.
- [23] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.
- [24] Torin Sandall and Tim Hinrichs. Policy-based access control with open policy agent. Conference presentations and project documentation, 2018. <https://www.openpolicyagent.org/>.
- [25] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium*, pages 1559–1575, 2021.
- [26] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.
- [27] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.
- [28] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.
- [29] Santiago Torres-Arias, Anish Kathpal, Hiten Gupta, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, pages 1393–1410, 2019.
- [30] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Use of Generative AI and Automated Tooling

The authors disclose the use of automated AI coding assistants for manuscript drafting assistance, LaTeX typography checking, and automated test harness refactoring via the paperloop framework. Specifically, LLM-powered tools assisted in organizing benchmark summaries, refactoring test harnesses, and formatting the double-blind review artifact. All references were subsequently verified by the authors against primary sources (publisher records or the live specification pages); an incorrect author list introduced in an earlier revision was found and corrected by that process. The core conceptual architecture, security proofs, empirical findings, and intellectual claims remain entirely the original work of the authors.

A Ethics and Dual-Use Analysis

The research presented in this paper introduces a defensive mechanism to ensure the security and provenance of autonomous agent releases. This work does not involve human subjects, does not analyze personally identifiable information, and does not conduct active measurements against production systems or networks operated by third parties. The 17 adversarial tamper vectors and the 1,075-profile corpus are fully synthetic and generated within isolated, local test environments for the sole purpose of evaluating the proposed cryptographic and policy enforcement mechanisms.

Dual-Use Considerations. Cryptographic execution attestation mechanisms, if repurposed maliciously, could theoretically be employed to construct invasive employee monitoring frameworks or enforce rigid software lock-in. EviAssure mitigates these dual-use concerns by enforcing decentralized multi-signature threshold governance and open policy schemas (`governance/release_policy.yaml`), and by keeping raw payloads out of evidence bundles entirely — traces carry digests only. Salted HMAC blinding (`blind_payload`) additionally unlinks the same recorded action across deployments, though as

Section 4.2 states, it does not conceal payload equality within one deployment.

Data Privacy & Parameter Blinding. Autonomous software agents frequently process confidential prompt texts, customer financial details, or proprietary source code snippets. To prevent public release audit logs from exposing sensitive data, EviAssure keeps raw payloads out of evidence bundles entirely—traces carry digests only—and the blinding protocol additionally re-keys those digests with a per-deployment salted HMAC (H_{blinded}), so the same recorded action is not linkable across environments. Within one environment the deterministic construction still reveals repetition (Definition 3). This preserves full Merkle tree inclusion auditability without storing proprietary inputs in the audit log.

Responsible Disclosure & Safety Impact. EviAssure raises the cost of forging or replaying release evidence produced by a compromised runner. It does *not* prevent prompt injection: an injected agent produces genuine actions that the collector honestly records and the gate correctly approves, and Section 7.8 measures this directly—all 75 anomalous corpus profiles, including the injection class, are APPROVED. The contribution is that such a release becomes *undeniable and attributable* after the fact, not that it is prevented. An earlier version of this appendix claimed prevention; that claim contradicted our own Section 7.8 results and has been withdrawn.

B Deployment Trade-offs

Two engineering consequences follow. *Audit granularity*: since a depth-bound proof costs 1.93 KB against 220,705 KB of raw trace, organisations can archive payloads to cold storage and keep only proofs online. *Replay state*: the gate must hold state across submissions, which V16 shows it cannot do across replicas without shared storage; a key-value store with atomic set-if-absent and TTL ΔT_{max} is the natural implementation, and it must fail closed when that store is unreachable rather than admit a split-brain replay. Neither deployment is measured here.

C Open Science Appendix

The reproducibility materials consist of the complete EviAssure implementation (cryptographic attestation, blinding, held-out inspection, and policy verification modules), the 1,075-profile agent execution trace corpus, the 17-vector tamper harness with negative controls, the executed DSSE/TUF/OPA baselines, production GitHub

Action workflows, benchmark scripts, and the paper figure generator. Furthermore, all datasets used to establish comparative baselines (e.g., the specific OPA Rego policies and JSON payloads used to evaluate the baseline) are included to guarantee full baseline reproducibility.

C.1 Anonymous Review Artifact & Verification

To comply with USENIX Security '27 Open Science guidelines while preserving double-blind review, the complete artifact has been published to a sanitized, anonymous repository at:

<https://anonymous.4open.science/r/eviassure-release-assurance/>

Additionally, an archived review package `eviassure_usenix27_artifact.zip` (size: 429.1 KB) has been prepared using `scripts/prepare_anonymous_artifact.py`. The archive carries SHA-256 digest:

7cf18a6643f5a1d605c6921b0f4daf10
69bb893b3cbf8e83e8c941767af441e0

C.2 Reproduction Workflow

To execute full reproduction of paper figures, benchmark results, and unit tests:

1. **Environment Setup:** Install dependencies via `pip install -e .` (Python ≥ 3.10).
2. **Unit & Integration Test Suite:** Run `pytest tests/ -v` (100 tests). The suite includes property-based crypto soundness tests, trusted-key-registry tests, and regression tests for every bypass fixed in the paper’s revision history; `test_docs_consistency.py` additionally binds the manuscript’s stated test count to the collected count, so the two cannot drift.
3. **Attestation Scaling & Throughput Microbenchmarks:** Run `python3 scripts/run_release_benchmark.py -repeats 5` to generate `results/benchmark_summary.json`.
4. **Adversarial & Baseline Evaluation:** Run `python3 scripts/run_security_eval.py --require-executed` to execute the 17-vector suite, the clean negative controls, the wire-fuzzing campaign, the executed DSSE/TUF/OPA baselines, the ablation matrix and the held-out inspection protocol into `results/security_evaluation.json`. These results are platform-independent. `--require-executed` fails rather than silently substituting a modeled baseline.

5. **Corpus Layered Evaluation:** Run `python3 scripts/generate_trace_corpus.py` then `python3 scripts/run_corpus_eval.py` to rebuild the 1,075-profile corpus and evaluate it through the integrity, policy and held-out inspection layers.
6. **Figure Generation & Manuscript PDF Rebuilding:** Run `python3 scripts/generate_paper_pdf.py` to regenerate paper plots and recompile `docs/usenix_paper_manuscript.pdf`.